Machine Learning

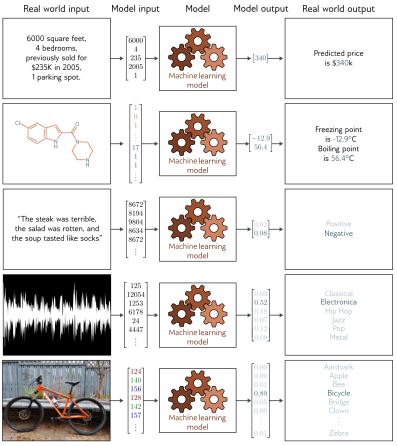
3. Ensemble Learning, Regularization; ML challenges

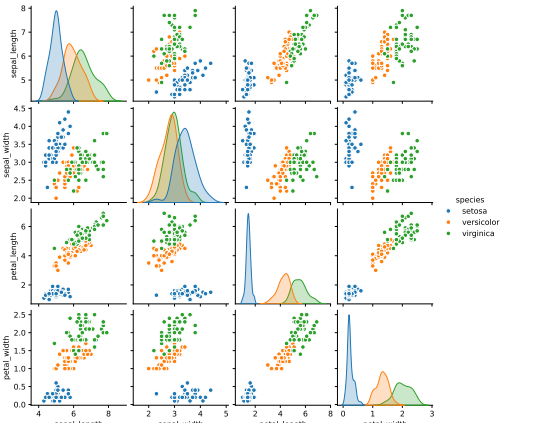
Louis Bagot
Université Claude Bernard Lyon 1

October 19, 2026

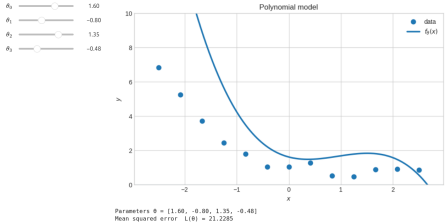
Where were we?

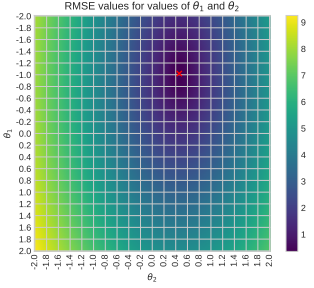
ML: Approximate a function from data examples $\{x_i, y_i\}$





Parametrized models $f_\theta(x_i) \approx y_i$ learn parameters θ to minimize loss: $\arg\min_\theta \mathcal{L}(\mathcal{D}, f_\theta)$





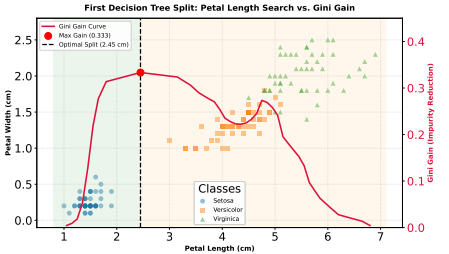
Where were we?
Adapting models & metrics for Classification

- Use a **softmax** to convert generic logits to probabilities
- Use the **CrossEntropy** loss with softmax to strongly punish errors
- Use **accuracy**, **recall**, **precision** to evaluate classif

Where were we?
Decision Trees

Recursively **split** the space along features:

- list all possible splits
  (interstices between datapoints)
- evaluate split quality
  (improvement in metric: Gini, MSE)
- take the best split; repeat



All models have strengths and weaknesses.
Imagine that we can use a bunch of those...

Can we combine their strengths?

All models have strengths and weaknesses.
Imagine that we can use a bunch of those...

Can we combine their strengths?
$\rightarrow$ this is *Ensemble Learning*!

All models have strengths and weaknesses.
Imagine that we can use a bunch of those...

Can we combine their strengths?
$\rightarrow$ this is *Ensemble Learning*!

Most competition-winning methods have been based on Ensembles.

All models have strengths and weaknesses.
Imagine that we can use a bunch of those...

Can we combine their strengths?
$\longrightarrow$ this is *Ensemble Learning*!

Most competition-winning methods have been based on Ensembles.

- critical question to keep in mind:
  why use M models instead of a single, M-times bigger model?

How do we formalize Ensemble Learning?

How do we formalize Ensemble Learning?

Assume we have M **base models** f_m.

Sometimes called *weak learners*: not very powerful, preventing overfit.

How do we formalize Ensemble Learning?

Assume we have M **base models** f_m.
Sometimes called *weak learners*: not very powerful, preventing overfit.

How do we train and use those?

How do we formalize Ensemble Learning?

Assume we have M **base models** f_m.
Sometimes called *weak learners*: not very powerful, preventing overfit.

How do we train and use those?

- $\rightarrow$ what input, output, model, loss function?
- $\rightrightarrows$ we will see *stacking*, *bagging* and *boosting*!

Can we just average them?

Can we just average them?

Simplest solution!

$$f(\boldsymbol{x}) = \frac{1}{M} \sum_{m=1}^{M} f_m(\boldsymbol{x})$$

Can we just average them?

Simplest solution!

$$f(\boldsymbol{x}) = \frac{1}{M} \sum_{m=1}^{M} f_m(\boldsymbol{x})$$

Potential issues:

Can we just average them?

Simplest solution!

$$f(\boldsymbol{x}) = \frac{1}{M} \sum_{m=1}^{M} f_m(\boldsymbol{x})$$

Potential issues:

- not all the same quality

Can we just average them?

Simplest solution!

$$f(\boldsymbol{x}) = \frac{1}{M} \sum_{m=1}^{M} f_m(\boldsymbol{x})$$

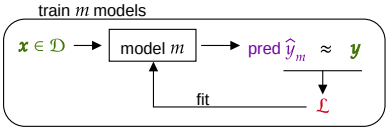
Potential issues:

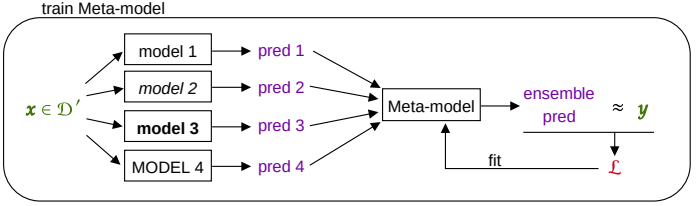
- not all the same quality
  - $\rightarrow$ *weigh* them? $w_m = 0$ removes bad models:

$$f(\boldsymbol{x}) = \sum_{m=1}^{M} w_m f_m(\boldsymbol{x})$$

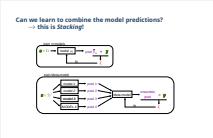
this is just a linear regression!

Can we learn to combine the model predictions?
$\rightarrow$ this is *Stacking*!





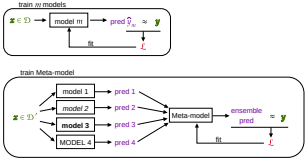
Notes for this slide



We first train our models independently (e.g. a linear model, a Decision Tree, we could use KNN or other methods as well...) Then we train a "Meta-model" which only sees (takes as input) the predictions of all the models and is trained to predict the label correctly. If the Meta-model is a linear regression, returns to the formula from the previous slide, where we learn the w_m.

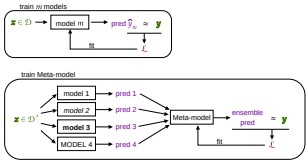
Note how 1/ the models are written differently, this is meant to represent different "families" of models: stacking gains from model diversity. 2/ the models and meta see different data ($\mathcal{D}$ and $\mathcal{D}'$). This is explained in the next slide!

When does Stacking work?



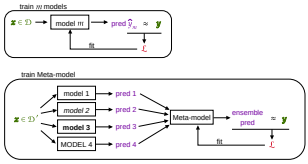
When does Stacking work?

- The more **diversity** in the models, the better!



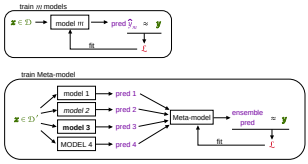
When does Stacking work?

- The more **diversity** in the models, the better!
  - correlation in errors **harm** the method!
  - $\rightarrow$ use *different families* of models



When does Stacking work?

- The more **diversity** in the models, the better!
  - correlation in errors **harm** the method!
  - $\rightarrow$ use *different families* of models
- Note, in the graphic, $\mathcal{D}$ and $\mathcal{D}'$:



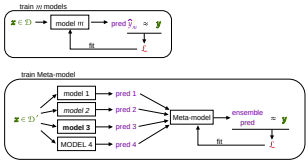
When does Stacking work?

- The more **diversity** in the models, the better!
  - correlation in errors **harm** the method!

  $\rightarrow$ use *different families* of models

- Note, in the graphic, $\mathcal{D}$ and $\mathcal{D}'$:
  - **do not** train models & Meta on the same data!

  $\rightarrow$ otherwise it will $\arg\max$!



Why not use a single, M-times bigger model?

Why not use a single, M-times bigger model?

Imagine I trained my model on a bunch of datasets
(from the same distribution underneath).

Why not use a single, M-times bigger model?

Imagine I trained my model on a bunch of datasets
(from the same distribution underneath).

- In average, does my model find the solution?

Why not use a single, M-times bigger model?

Imagine I trained my model on a bunch of datasets
(from the same distribution underneath).

- In average, does my model find the solution?

- How different are solutions found?

Why not use a single, M-times bigger model?

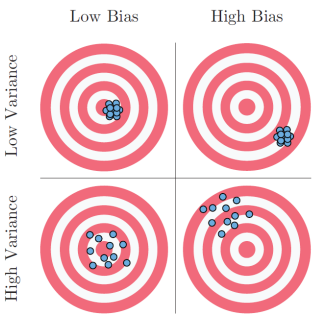
Imagine I trained my model on a bunch of datasets
(from the same distribution underneath).

- In average, does my model find the solution?
  - Notion of **bias**.
    linear: biased if relationship is not linear.
- How different are solutions found?

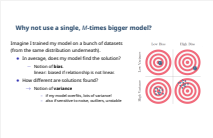
Why not use a single, *M*-times bigger model?

Imagine I trained my model on a bunch of datasets (from the same distribution underneath).

- In average, does my model find the solution?
  - Notion of **bias**.
    linear: biased if relationship is not linear.
- How different are solutions found?
  - → Notion of **variance**
    - if my model overfits, lots of variance!
    - also if sensitive to noise, outliers, unstable



Notes for this slide



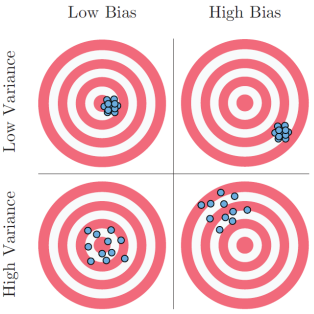
As notions related to variance, you may also hear of **stability** (sensitivity to tiny changes in the data) or **robustness** (sensitivity to outliers, noise, shift).

These concepts tend to have different definitions depending on the exact setting and problem of interest. Variance is a bit more general term, and considers the variations *in expectation* (so over many, many datasets) while stability and robustness tend to describe more specific, well-chosen "attacks" that might mess up the model.

Why not use a single, M-times bigger model?

Notion of **bias-variance trade-off**:

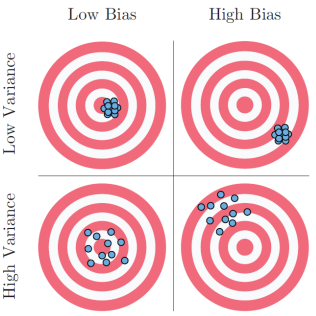
- Simple model: biased, low variance
- Complicated model: low bias, high variance



Why not use a single, *M*-times bigger model?

Notion of **bias-variance trade-off**:

- Simple model: biased, low variance
- Complicated model: low bias, high variance
  - risk with the *M*-times bigger model!



Notes for this slide

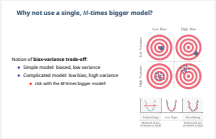


Image adapted from the ML knowledge base.

Mathematically we can show that the total error committed is (see book p 161)

$$\text{total error} = \text{bias}^2 + \text{variance} + \text{noise}$$

So we can decide to trade a little bit of variance for a little bit of bias, one way or the other. (the noise may come from noise in the data which we cannot compress/learn)

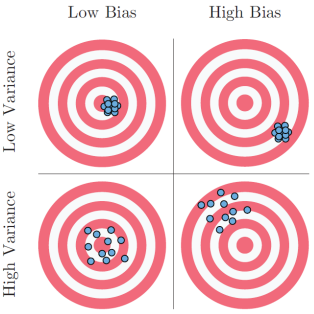
The main issue with models with a lot of variance is that, if I have a new problem and I fit the model on my data, it's hard to know if I got lucky or not!

Why not use a single, M-times bigger model?

Notion of **bias-variance trade-off**:

- Simple model: biased, low variance
- Complicated model: low bias, high variance
  - risk with the M-times bigger model!

→ **Stacking reduces variance**

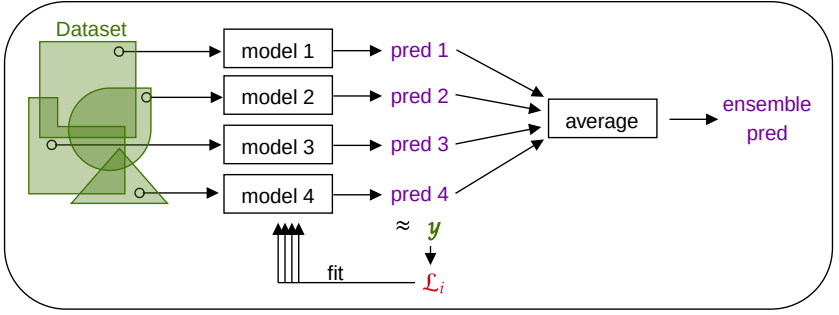


Reduce variance by training on sub-sets of the dataset?

Reduce variance by training on sub-sets of the dataset?

$\rightarrow$ **this is *Bagging*!**

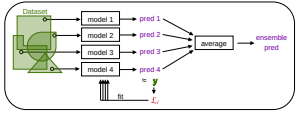
Reduce variance by training on sub-sets of the dataset?
$\rightarrow$ this is *Bagging*!



Reduce variance by training on sub-sets of the dataset?
$\rightarrow$ this is *Bagging*!

Stacking: diversity through different *model families*

Bagging: diversity through **the data seen!**

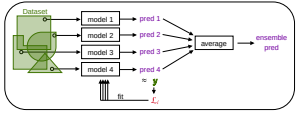


Reduce variance by training on sub-sets of the dataset?
$\rightarrow$ this is *Bagging*!

Stacking: diversity through different *model families*

Bagging: diversity through **the data seen!**

- Often with the same family of models
  - $\rightarrow$ less worries of good/bad models

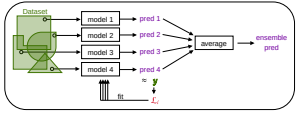


Reduce variance by training on sub-sets of the dataset?
$\rightarrow$ this is *Bagging*!

Stacking: diversity through different *model families*

Bagging: diversity through **the data seen!**

- Often with the same family of models
  - $\rightarrow$ less worries of good/bad models
- Sub-datasets sampled through *bootstrapping*: sample N times with replacement



Notes for this slide



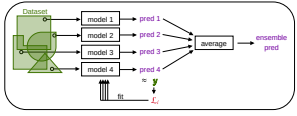
About bootstrapping: in average, for a dataset of size N, sampling N points with replacement (so we can sample the same point twice) means we only sample around 63% of the data. So the models will see quite different data!

Reduce variance by training on sub-sets of the dataset?
$\rightarrow$ this is *Bagging*!

Stacking: diversity through different *model families*

Bagging: diversity through **the data seen!**

- Often with the same family of models
  - $\rightarrow$ less worries of good/bad models
- Sub-datasets sampled through *bootstrapping*: sample N times with replacement
  - $\rightarrow$ simulating different datasets!



What model family to use for bagging?

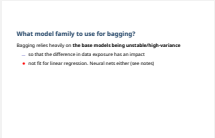
Bagging relies heavily on **the base models being unstable/high-variance**

- ... so that the difference in data exposure has an impact

What model family to use for bagging?

Bagging relies heavily on **the base models being unstable/high-variance**

- ... so that the difference in data exposure has an impact
- not fit for linear regression. Neural nets either (see notes)



Notes for this slide

Neural networks were found to just under-perform with less data, but not with the kind of instabilities that let bagging work. The bootstrapping (which shows some samples several times and others none) was found not to impact NNs too much, more data was always better (see Why aren't Bootstrapped Neural Networks better?)

What model family to use for bagging?

Bagging relies heavily on **the base models being unstable/high-variance**

- ... so that the difference in data exposure has an impact
- not fit for linear regression. Neural nets either (see notes)

→ **Decision Trees** are the main character for bagging (and a lot of Ensemble)

What model family to use for bagging?

Bagging relies heavily on **the base models being unstable/high-variance**

- ... so that the difference in data exposure has an impact
- not fit for linear regression. Neural nets either (see notes)

→ **Decision Trees** are the main character for bagging (and a lot of Ensemble)

Bagging with Decision Trees: this is just **Random Forests!**

What model family to use for bagging?

Bagging relies heavily on **the base models being unstable/high-variance**

- ... so that the difference in data exposure has an impact
- not fit for linear regression. Neural nets either (see notes)

→ **Decision Trees** are the main character for bagging (and a lot of Ensemble)

Bagging with Decision Trees: this is just **Random Forests!**

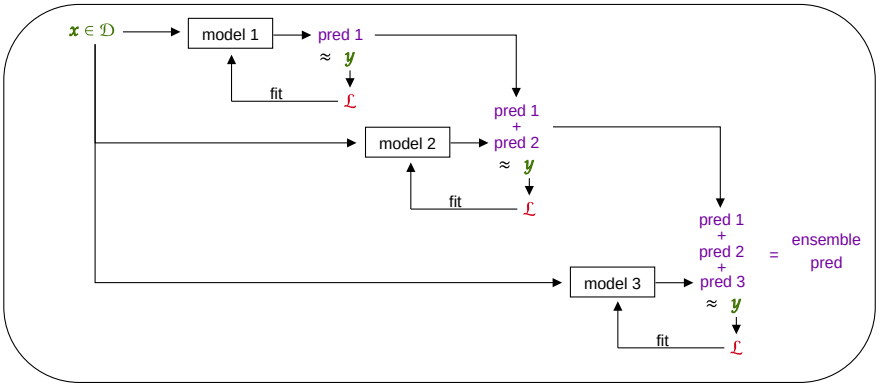
→ enhanced Bagging idea by also taking random features to split over

Can we refine the estimate of the previous model?

Can we refine the estimate of the previous model?

$\rightarrow$ this is *Boosting*!

Can we refine the estimate of the previous model?
$\rightarrow$ this is *Boosting*!

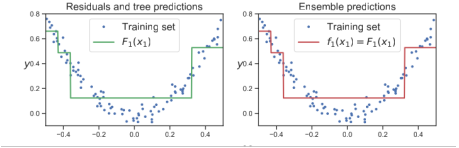


Can we refine the estimate of the previous model?
$\rightarrow$ this is *Boosting*!

Try to predict the

error committed by the model so far:

$$f_1(\boldsymbol{x}) \approx \boldsymbol{y}$$

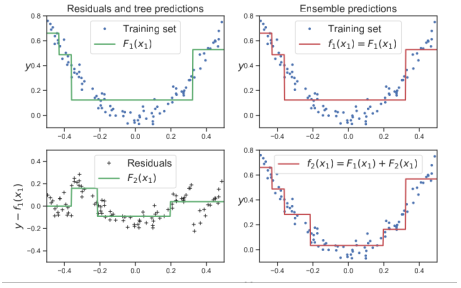


Can we refine the estimate of the previous model? $\rightarrow$ this is *Boosting*!

Try to predict the

error committed by the model so far:

$$f_1(\boldsymbol{x}) \approx \boldsymbol{y}$$
$$f_2(\boldsymbol{x}) \approx \boldsymbol{y} - f_1(\boldsymbol{x})$$

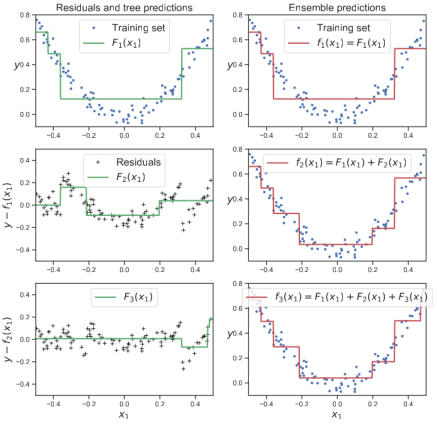


Can we refine the estimate of the previous model?
$\rightarrow$ this is *Boosting*!

Try to predict the

error committed by the model so far:

$$f_1(\boldsymbol{x}) \approx \boldsymbol{y}$$
$$f_2(\boldsymbol{x}) \approx \boldsymbol{y} - f_1(\boldsymbol{x})$$
$$f_3(\boldsymbol{x}) \approx \boldsymbol{y} - (f_1(\boldsymbol{x}) + f_2(\boldsymbol{x}))$$

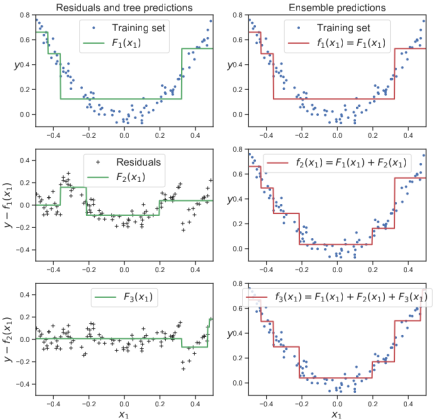


Can we refine the estimate of the previous model?
$\rightarrow$ this is *Boosting*!

Try to predict the

error committed by the model so far:

$$
\begin{aligned}
f_1(\boldsymbol{x}) &\approx \boldsymbol{y} \\
f_2(\boldsymbol{x}) &\approx \boldsymbol{y} - f_1(\boldsymbol{x}) \\
f_3(\boldsymbol{x}) &\approx \boldsymbol{y} - (f_1(\boldsymbol{x}) + f_2(\boldsymbol{x})) \\
&\vdots \\
f_m(\boldsymbol{x}) &\approx \boldsymbol{y} - \sum_{i<m} f_i(\boldsymbol{x})
\end{aligned}
$$



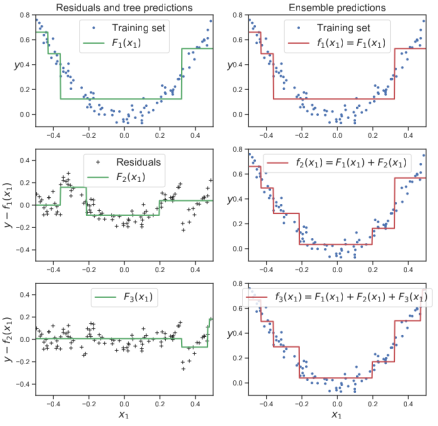
Can we refine the estimate of the previous model?
$\rightarrow$ this is *Boosting*!

Try to predict the

error committed by the model so far:

$$f_1(\boldsymbol{x}) \approx \boldsymbol{y}$$
$$f_2(\boldsymbol{x}) \approx \boldsymbol{y} - f_1(\boldsymbol{x})$$
$$f_3(\boldsymbol{x}) \approx \boldsymbol{y} - (f_1(\boldsymbol{x}) + f_2(\boldsymbol{x}))$$
$$\vdots$$
$$f_m(\boldsymbol{x}) \approx \boldsymbol{y} - \sum_{i<m} f_i(\boldsymbol{x})$$

$$f(\boldsymbol{x}) = \sum_{m=1}^{M} f_m(\boldsymbol{x})$$



Notes for this slide

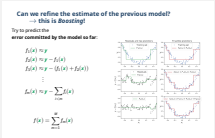


Image from the Proba ML book page 613. (They have a slightly different notation convention, where f_m "the whole model up to m, so it corresponds to the sum, and they use F_i to describe individual models, but I found this more confusing than using f_m for the individual models as we've been doing so far.)

Note that

$$\begin{aligned} f_2(\boldsymbol{x}) &\approx \boldsymbol{y} - f_1(\boldsymbol{x}) \\ \iff f_1(\boldsymbol{x}) + f_2(\boldsymbol{x}) &\approx \boldsymbol{y} \end{aligned}$$

So the prediction is the sum of models; from another perspective, each model *adds* a little correction to the previous prediction, hence why the slide shows a sum of predictions.

Boosting: how does it behave?

Stacking and Bagging train independent models in parallel.

Boosting trains models **sequentially**!

- Not as computationally efficient
- Boosting **reduces bias** more than variance
  - often stronger than random forests for trees
  - more susceptible to overfit!
- see the notebook "`tree_vs_ensemble`" to play with it

Adding predictions for regression: OK. Classif though?

- If I use sigmoid$(f_1(\boldsymbol{x})) \approx \boldsymbol{y}$, how do I train f_2?
  - $\rightarrow$ I'd like my final prediction to be sigmoid$(f_1(\boldsymbol{x}) + f_2(\boldsymbol{x})) \approx \boldsymbol{y}$...
    - but then what's my link between f_1 and f_2?

Adding predictions for regression: OK. Classif though?

- If I use sigmoid$(f_1(\boldsymbol{x})) \approx \boldsymbol{y}$, how do I train f_2?
  - $\rightarrow$ I'd like my final prediction to be sigmoid$(f_1(\boldsymbol{x}) + f_2(\boldsymbol{x})) \approx \boldsymbol{y}$...
    - but then what's my link between f_1 and f_2?

- Observation: $f_2(\boldsymbol{x}) \approx \boldsymbol{y} - f_1(\boldsymbol{x})$ is the **negative gradient** of MSE!

$$-\frac{\partial(\boldsymbol{y} - f_1(\boldsymbol{x}))^2}{\partial f_1(\boldsymbol{x})} = \boldsymbol{y} - f_1(\boldsymbol{x}) \approx f_2(\boldsymbol{x})$$

  - $\rightarrow$ answers "how should I change $f_1(\boldsymbol{x})$ to be MSE-closer to $\boldsymbol{y}$? "
  - $\rightrightarrows$ great for regression with MSE! ... not others!

What update rule to use then?

What update rule to use then?

Through their gradients,
different losses incur different update rules from one model to the next.

- $\rightarrow$ This is **Gradient Boosting**!
- $\rightrightarrows$ it **generalizes** the intuitive “refine the prediction” to any loss

What update rule to use then?

Through their gradients,

different losses incur different update rules from one model to the next.

→ This is **Gradient Boosting**!

⇉ it **generalizes** the intuitive "refine the prediction" to any loss

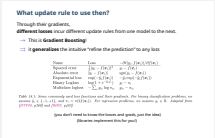
Name	Loss	$-\partial\ell(y_i, f(\boldsymbol{x}_i))/\partial f(\boldsymbol{x}_i)$
Squared error	$\frac{1}{2}(y_i - f(\boldsymbol{x}_i))^2$	$y_i - f(\boldsymbol{x}_i)$
Absolute error	$\lvert y_i - f(\boldsymbol{x}_i)\rvert$	$\operatorname{sgn}(y_i - f(\boldsymbol{x}_i))$
Exponential loss	$\exp(-\tilde{y}_i f(\boldsymbol{x}_i))$	$-\tilde{y}_i \exp(-\tilde{y}_i f(\boldsymbol{x}_i))$
Binary Logloss	$\log(1 + e^{-\tilde{y}_i f_i})$	$y_i - \pi_i$
Multiclass logloss	$-\sum_c y_{ic} \log \pi_{ic}$	$y_{ic} - \pi_{ic}$

Table 18.1: Some commonly used loss functions and their gradients. For binary classification problems, we assume $\tilde{y}_i \in \{-1, +1\}$, and $\pi_i = \sigma(2f(\boldsymbol{x}_i))$. For regression problems, we assume $y_i \in \mathbb{R}$. Adapted from [HTF09, p360] and [BH07, p483].

(you don't need to know the losses and grads, just the idea)

(libraries implement this for you!)

Notes for this slide



Screenshot from the Proba ML Book, p 617.

Can we do something about Boosting overfitting?

Can we do something about Boosting overfitting?

General rule of Machine Learning:

Use *regularization* to control complicated models

Can we do something about Boosting overfitting?

General rule of Machine Learning:

Use *regularization* to control complicated models

Regularization: any process that forces the model to find simpler solutions.

- **The** main ML approach to fighting overfit
- Can be seen as trading some bias for variance

Can we do something about Boosting overfitting?

General rule of Machine Learning:

Use *regularization* to control complicated models

Regularization: any process that forces the model to find simpler solutions.

- **The** main ML approach to fighting overfit
- Can be seen as trading some bias for variance
- Main method: **add a penalty term to the loss/objective** for complexity

Regularization: what penalty term to use?

Regularization: what penalty term to use?

General observation: fitting specific datapoints requires **high weights/values**

$\rightarrow$ penalize high weights by adding their magnitude to the loss!

$$\arg\min_{\boldsymbol{\theta}} \mathcal{L}(\mathcal{D}, f_{\boldsymbol{\theta}}) + \lambda \sum_{i=1}^{\text{n_param}} {\theta_i}^2$$

$\rightrightarrows$ each weight θ_i is encouraged to be small

- λ controls the balance "solving the objective $\leftrightarrow$ model complexity"

Regularization: what penalty term to use?

- $\sum_{i=1}^{\text{n_param}} {\theta_i}^2 \doteq \|\boldsymbol{\theta}\|_2^2$ – This is the L2 norm of vector $\boldsymbol{\theta}$:
  - this is **L2 regularization** or **Ridge regression** (for linear models)
  - more strongly penalizes higher parameters; difficult to reach 0 exactly

Regularization: what penalty term to use?

- $\sum_{i=1}^{\text{n_param}} \theta_i^2 \doteq \|\boldsymbol{\theta}\|_2^2$ – This is the L2 norm of vector $\boldsymbol{\theta}$:
  - this is **L2 regularization** or **Ridge regression** (for linear models)
  - more strongly penalizes higher parameters; difficult to reach 0 exactly
- $\sum_{i=1}^{\text{n_param}} |\theta_i| \doteq \|\boldsymbol{\theta}\|_1$ – This is the L1 norm of vector $\boldsymbol{\theta}$:
  - this is **L1 regularization** or **Lasso regression** (for linear models)
  - less strongly penalizes higher parameters; easier to reach 0 exactly

Regularization: what penalty term to use?

- $\sum_{i=1}^{\text{n_param}} {\theta_i}^2 \doteq \|\boldsymbol{\theta}\|_2^2$ – This is the L2 norm of vector $\boldsymbol{\theta}$:
  - this is **L2 regularization** or **Ridge regression** (for linear models)
  - more strongly penalizes higher parameters; difficult to reach 0 exactly
- $\sum_{i=1}^{\text{n_param}} |\theta_i| \doteq \|\boldsymbol{\theta}\|_1$ – This is the L1 norm of vector $\boldsymbol{\theta}$:
  - this is **L1 regularization** or **Lasso regression** (for linear models)
  - less strongly penalizes higher parameters; easier to reach 0 exactly
- "elastic net" regularization does L1+L2
- **!! magnitude of linear weights depends on the magnitude of variables !!**
  - → **normalize**!!

Ridge/Lasso: why use Regularization for linear models?

Since they don’t really overfit..?

Ridge/Lasso: why use Regularization for linear models?

Since they don't really overfit..?

- They **can** overfit! Depends on the features (see next slide!)
- Regularization can help **remove useless features**:
  - if two features are strongly correlated (e.g., weight and #items or cm and inch): force one of them down
  - if a feature is useless: lasso can make its weight exactly 0
    - $\rightarrow$ form of **feature selection**!

Regularization: what does it look like?

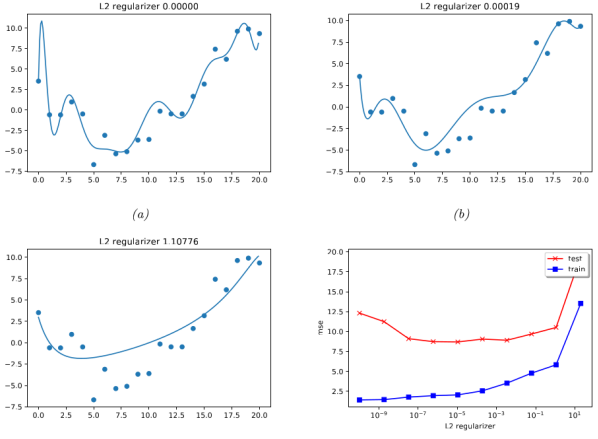


Figure 4.5: (a-c) Ridge regression applied to a degree 14 polynomial fit to 21 datapoints. (d) MSE vs strength of regularizer. The degree of regularization increases from left to right, so model complexity decreases from left to right. Generated by linreg_poly_ridge.ipynb.

Notes for this slide



Plot taken from the Proba ML Book, p 124.

Linear models can definitely overfit if the features themselves are over-engineered! Here, the features are the powers of x, so the linear model ends up putting a lot of weight on the features associated with high powers to fit all the points it can. Regularization forces those weights down, forcing a “simpler” solution.

Regularization: what about Decision Trees?

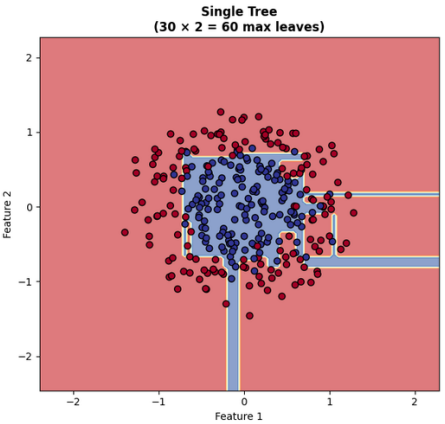
Not much sense to punish the split's feature value or the leaf value...

Regularization: what about Decision Trees?

Not much sense to punish the split's feature value or the leaf value...

Tree complexity controlled by
number of leaves and tree depth

- → penalize those!
- ⇉ prevent splits with too little perf increase



Regularization: what about Decision Trees?

In the context of Tree Gradient Boosting specifically

- consider each added tree's node as a "small correction"
  - $\rightarrow$ we want to prevent huge, unjustified corrections
  - $\rightrightarrows$ leaf values can be dampened by a factor!

Regularization: what about Decision Trees?

In the context of Tree Gradient Boosting specifically

- consider each added tree's node as a "small correction"
  - $\rightarrow$ we want to prevent huge, unjustified corrections
  - $\rightrightarrows$ leaf values can be dampened by a factor!

Decision Trees + Gradient Boosting + Regularizations + learning rate + more
= XGBoost, clear contender for "the best"

- Won too many competitions!
- Method described in a paper, XGBoost: A Scalable Tree Boosting System
- Library maintained by the community: xgboost
- start from the mean, add a few (3-6) shallow trees

Summary, please?

- **Ensemble Learning** combines models.
  - *Stacking*: learn to combine predictions
    - ▶ independent, diverse models from different families
    - ● more problem-specific than the others, no general algo
  - *Bagging*: generate diverse datasets
    - ▶ independent models from the same family, diverse data
    - ▶ Random Forest as the main interesting version
    - → balances out the tree's instability: variance reduction
    - ● perf is OKay-ish
  - *Boosting*: keep adjusting model predictions
    - ● reduces bias; great perf
    - ● more prone to overfitting
    - ▶ XGBoost MVP

Summary, please?

- **Ensemble Learning** combines models.
  - *Stacking*: learn to combine predictions
  - *Bagging*: generate diverse datasets
  - *Boosting*: keep adjusting model predictions

- **Regularization** addresses overfit
  - Prevents models from finding complicated solutions
  - Generally a penalty term on the loss (weight size, number of leaves)
  - Critical from linear models to neural nets, through trees!

Regularization: how do I pick my loss weight λ?

We have to tune it...

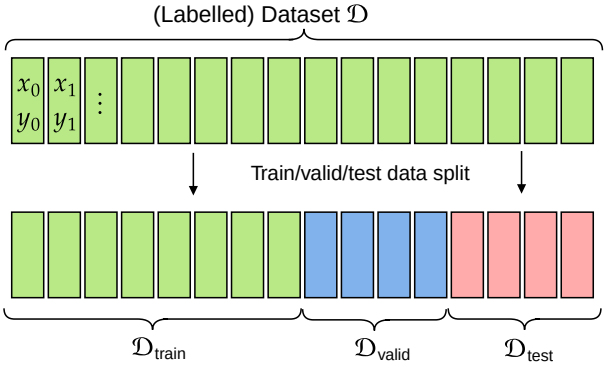
Hyper-parameter: any value that we pick and is not optimized by the model's algo

- KNN's k, number of neighbors
- Tree depth/max number of leaves
- Temperature for softmax (controls how "flat" it is)
- Number of models in Ensemble
- Regularization factors...
- ... and a lot more to come!
  - complicated methods tend to have more hyper-parameters...

How do I pick my hyper-parameters?

DO NOT optimize them against your test set!!

→ you're just going to find the best value *for your test set*.
instead, further break down your dataset:



How do I pick my hyper-parameters?

- For each hyper-parameter choice, **train** on the train set
- **Optimize** your hyper-parameters on the validation set
- **Evaluate** your model on the test set

How do I pick my hyper-parameters?

- For each hyper-parameter choice, **train** on the train set
- **Optimize** your hyper-parameters on the validation set
- **Evaluate** your model on the test set

$\rightarrow$ OK, but how do I "optimize"?

- Good old: try a bunch of intuitive values by hand
- Grid search: try all of them within a window, e.g., $\lambda \in [0, 0.5, 1, 2]$
  - (values to try) (number of hyper-parameters)
- Smarter optimizers: see the course on MLOps!

$\rightarrow$ no matter how you do it, it's a pain!

Anything more I need to know?

Machine Learning is huge; lots more

- models (genetic algos, SVM, KDE...)
- problems (imbalance, anomaly detection, feature extraction, explainability...)
- solutions!

$\rightarrow$ impossible to cover in 10 or 30 hours!

Anything more I need to know?

Machine Learning is huge; lots more

- models (genetic algos, SVM, KDE...)
- problems (imbalance, anomaly detection, feature extraction, explainability...)
- solutions!

$\rightarrow$ impossible to cover in 10 or 30 hours!

Point of this course:

Give you the tools to understand any problem, any method.

- All methods have advantages/disadvantages
- Always ask "why not do this?" "how would I do it?"
  - $\rightarrow$ (that's why all titles have been questions :D)
  - — **trust** in your ability to dive in, read docs, understand

Where to next?

Deep Learning course!

Where to next?

Deep Learning course!

- *Neural Networks* are one specific parametrized model
- Works really well on huge amounts of hugely complicated data
  - image
  - text
  - sound...

Where to next?

Deep Learning course!

- *Neural Networks* are one specific parametrized model
- Works really well on huge amounts of hugely complicated data
  - image
  - text
  - sound...
- "Traditional" ML methods are still relevant.
  - Linear relationship: nothing beats a linear regression.
  - Decision Trees can be read & visualized
  - For data of smaller dimension, Random Forests / XGBoost still win vs NNs.